

Assignment-12 (Lab-12.2)

Course : AI Assisted Coding

Topic:- Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Name : Batti Chandan Singh

Hall Ticket No : 2303A51515

Batch : 22

Task-01

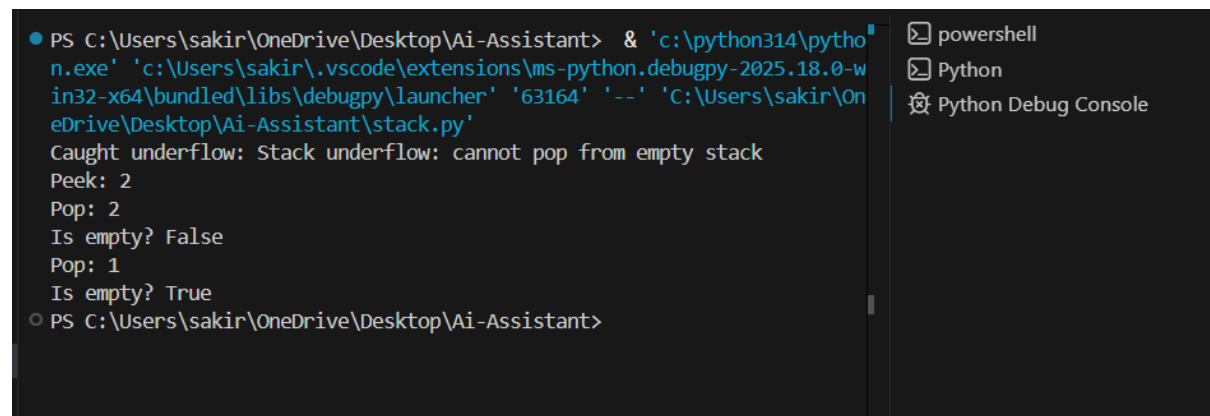
Final Optimal Prompt:

take 1task
and write a program python to implement stack with operations are
push,pop,peek,is_empty
and handle error of stack underflow

Code Screenshot :

```
1 class StackUnderflowError(Exception):
2     pass
3 class Stack:
4     """Simple stack implementation with push, pop, peek, is_empty."""
5
6     def __init__(self):
7         self._items = []
8
9     def push(self, item):
10        self._items.append(item)
11
12    def pop(self):
13        if self.is_empty():
14            raise StackUnderflowError("Stack underflow: cannot pop from empty stack")
15        return self._items.pop()
16
17    def peek(self):
18        if self.is_empty():
19            raise StackUnderflowError("Stack underflow: cannot peek from empty stack")
20        return self._items[-1]
21
22    def is_empty(self):
23        return len(self._items) == 0
24
25    def __len__(self):
26        return len(self._items)
27
28    def __repr__(self):
29        return f"Stack({self._items})"
30
31 if __name__ == "__main__":
32     s = Stack()
33     try:
34         s.pop()
35     except StackUnderflowError as e:
36         print("Caught underflow:", e)
37
38     s.push(1)
39     s.push(2)
40     print("Peek:", s.peek())
41     print("Pop:", s.pop())
42     print("Is empty?", s.is_empty())
43     print("Pop:", s.pop())
44     print("Is empty?", s.is_empty())
```

Output Screenshot:



```
● PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63164' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\stack.py'
Caught underflow: Stack underflow: cannot pop from empty stack
Peek: 2
Pop: 2
Is empty? False
Pop: 1
Is empty? True
○ PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

After generating the Stack program using AI, I understood how a stack data structure works internally using Python classes. AI helped ensure that every method had a clear docstring, making the code easy to understand. While testing, I observed how push adds elements to the top, pop removes the most recent element, and peek only views the top element without removing it. The AI's error handling for underflow helped me learn how to prevent popping from an empty stack. Overall, I understood the LIFO (Last In First Out) nature clearly through execution.

Task-02

Final Optimal Prompt:

write a python program of linear search and binary search with documented string in every function

Code Screenshot :

```

1 from typing import List, Optional
2 def linear_search(arr: List[int], target: int) -> int:
3     """Perform a linear search for ``target`` in ``arr``.
4     Args:
5         arr: A list of integers to search through.
6         target: The integer value to find.
7     Returns:
8         The index of the first occurrence of ``target`` in ``arr``.
9         Returns ``-1`` if ``target`` is not found.
10    Complexity:
11        Time: O(n), Space: O(1)
12    """
13    for i, val in enumerate(arr):
14        if val == target:
15            return i
16    return -1
17 def binary_search_iterative(arr: List[int], target: int) -> int:
18     """Perform an iterative binary search for ``target`` in a sorted ``arr``.
19     Args:
20         arr: A list of integers sorted in non-decreasing order.
21         target: The integer value to find.
22     Returns:
23         The index of ``target`` in ``arr`` if present, otherwise ``-1``.
24     Notes:
25         The list *must* be sorted. Behaviour is undefined for unsorted input.
26    Complexity:
27        Time: O(log n), Space: O(1)
28    """
29    left, right = 0, len(arr) - 1
30    while left <= right:
31        mid = (left + right) // 2
32        if arr[mid] == target:
33            return mid
34        if arr[mid] < target:
35            left = mid + 1
36        else:
37            right = mid - 1
38    return -1
39 def binary_search_recursive(arr: List[int], target: int, left: int = 0, right: Optional[int] = None) -> int:
40     """Recursive binary search helper.
41     Args:
42         arr: A list of integers sorted in non-decreasing order.
43         target: The integer value to find.
44         left: Left index of the current search range (inclusive).
45         right: Right index of the current search range (inclusive). If ``None``,
46             it will be set to ``len(arr) - 1``.
47     Returns:
48         The index of ``target`` in ``arr`` if present, otherwise ``-1``.
49    Complexity:
50        Time: O(log n), Space: O(log n) due to recursion depth.
51    """
52    if right is None:
53        right = len(arr) - 1
54    if left > right:
55        return -1
56    mid = (left + right) // 2
57    if arr[mid] == target:
58        return mid
59    if arr[mid] < target:
60        return binary_search_recursive(arr, target, mid + 1, right)
61    return binary_search_recursive(arr, target, left, mid - 1)
62 def _demo() -> None:
63     """Run a small demonstration and basic checks for the search functions."""
64     data = [1, 3, 5, 7, 9, 11]
65     targets = [7, 2, 11, 1]
66     print("Array:", data)
67     for t in targets:
68         li = linear_search(data, t)

```

Output Screenshot:

```

'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle
d\libs\debugpy\launcher' '55862' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\searches.py'
Array: [1, 3, 5, 7, 9, 11]
Target=7: linear=3, binary_iter=3, binary_rec=3
Target=2: linear=-1, binary_iter=-1, binary_rec=-1
Target=11: linear=5, binary_iter=5, binary_rec=5
Target=1: linear=0, binary_iter=0, binary_rec=0
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

After running both search algorithms, AI-generated code helped me understand the difference in execution flow. Linear Search checks each element one by one, which works on any unsorted list. Binary Search, however, only worked after I sorted the list, showing it

needs ordered data. Running tests on small and large arrays made the performance gap very visible — binary search was always much faster. The AI complexity explanation made it clear why Linear Search is $O(n)$ and Binary Search is $O(\log n)$. This task helped me truly compare where each algorithm is suitable.

Task-03

Final Optimal Prompt:

write a python program to test driven development on simple calculator
and document the process with comments.

Code Screenshot :

```

50
51 # Step 1: Write a failing test for the calculator's addition function
52 def test_addition():
53     # This test will fail because we haven't implemented the add function yet
54     assert add(2, 3) == 5
55 # Step 2: Implement the add function to make the test pass
56 def add(a, b):
57     return a + b
58 # Step 3: Run the test to ensure it passes
59 test_addition()
60 print("Addition test passed!")
61 # Step 4: Write a failing test for the calculator's subtraction function
62 def test_subtraction():
63     # This test will fail because we haven't implemented the subtract function yet
64     assert subtract(5, 2) == 3
65 # Step 5: Implement the subtract function to make the test pass
66 def subtract(a, b):
67     return a - b
68 # Step 6: Run the test to ensure it passes
69 test_subtraction()
70 print("Subtraction test passed!")
71 # Step 7: Write a failing test for the calculator's multiplication function
72 def test_multiplication():
73     # This test will fail because we haven't implemented the multiply function yet
74     assert multiply(4, 3) == 12
75 # Step 8: Implement the multiply function to make the test pass
76 def multiply(a, b):
77     return a * b
78 # Step 9: Run the test to ensure it passes
79 test_multiplication()
80 print("Multiplication test passed!")
81 # Step 10: Write a failing test for the calculator's division function
82 def test_division():
83     # This test will fail because we haven't implemented the divide function yet
84     assert divide(10, 2) == 5
85 # Step 11: Implement the divide function to make the test pass
86 def divide(a, b):
87     if b == 0:
88         raise ValueError("Cannot divide by zero")
89     return a / b
90 # Step 12: Run the test to ensure it passes
91 test_division()
92 print("Division test passed!")
93

```

Output Screenshot:

```

PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c;; cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '65133' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\stack.py'
Addition test passed!
Subtraction test passed!
Multiplication test passed!
Division test passed!
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

Writing tests first before writing the actual calculator function helped me understand how TDD works in real projects. At first, the tests failed since the functions were not implemented yet. After asking AI to create the implementation functions, all tests passed successfully. This showed how TDD ensures code correctness from the beginning and forces developers to think about requirements before writing logic. The process also helped me understand how AI can help generate unit tests and valid implementations quickly

Task-04

Final Optimal Prompt:

Write a python program of queue the methods are enqueue, dequeue, front, is_empty and handle the overflow , underflow
include docstrings for every method

Code Screenshot :

```
1 from typing import Optional
2 class QueueOverflowError(Exception):
3     """Raised when an enqueue is attempted on a full queue."""
4 class QueueUnderflowError(Exception):
5     """Raised when a dequeue or front is attempted on an empty queue."""
6 class Queue:
7     """A simple FIFO queue with optional fixed capacity.
8     Methods
9     """
10    enqueue(item): Add 'item' to the back of the queue.
11    dequeue(): Remove and return the item at the front of the queue.
12    front(): Return the item at the front without removing it.
13    is_empty(): Return True if the queue has no items.
14    The queue supports an optional 'max_size'. If 'max_size' is provided,
15    'enqueue' will raise 'QueueOverflowError' when the queue is full.
16    """
17    def __init__(self, max_size: Optional[int] = None) -> None:
18        """Create a new queue.
19        Args:
20            max_size: Optional maximum number of items allowed in the queue.
21            If 'None', the queue is unbounded.
22        """
23        if max_size is not None and max_size <= 0:
24            raise ValueError("max_size must be None or a positive integer")
25        self._items = []
26        self.max_size = max_size
27    def enqueue(self, item) -> None:
28        """Add 'item' to the back of the queue.
29        Raises:
30            QueueOverflowError: If the queue has reached 'max_size'.
31        """
32        if self.max_size is not None and len(self._items) >= self.max_size:
33            raise QueueOverflowError("Queue overflow: cannot enqueue to full queue")
34        self._items.append(item)
35    def dequeue(self):
36        """Remove and return the item at the front of the queue.
37        Raises:
38            QueueUnderflowError: If the queue is empty.
39        """
40        if self.is_empty():
41            raise QueueUnderflowError("Queue underflow: cannot dequeue from empty queue")
42        return self._items.pop(0)
43    def front(self):
44        """Return (but do not remove) the front item of the queue.
45        Raises:
46            QueueUnderflowError: If the queue is empty.
47        """
48        if self.is_empty():
49            raise QueueUnderflowError("Queue underflow: cannot access front of empty queue")
50        return self._items[0]
51    def is_empty(self) -> bool:
52        """Return True if the queue contains no items, otherwise False."""
53        return len(self._items) == 0
54    def __len__(self) -> int: # convenience
55        return len(self._items)
56    def __repr__(self) -> str:
57        return f"Queue({self._items}, max_size={self.max_size})"
58    def _demo() -> None:
59        """Demonstrate 'Queue' usage and error handling."""
60        q = Queue(max_size=3)
61        print("Initial:", q)
62        try:
63            q.dequeue()
64        except QueueUnderflowError as e:
65            print("Caught underflow:", e)
66        q.enqueue("a")
67        q.enqueue("b")
68        q.enqueue("c")
```

Output Screenshot:

```
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:: cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57301' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\queue.py'

Initial: Queue([], max_size=3)
Caught underflow: Queue underflow: cannot dequeue from empty queue
After enqueues: Queue(['a', 'b', 'c'], max_size=3)
Caught overflow: Queue overflow: cannot enqueue to full queue
Front: a
Dequeue: a
After one dequeue: Queue(['b', 'c'], max_size=3)
Dequeue: b
Dequeue: c
Caught underflow on front: Queue underflow: cannot access front of empty queue
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

After generating and running the Queue implementation, I clearly understood how a queue follows FIFO (First In First Out). The AI-generated docstrings and error handling made the code easy to follow. Enqueue added elements to the rear of the queue, while dequeue removed from the front. When I tried to dequeue from an empty queue, the error handling triggered correctly, teaching me how underflow conditions must be managed. Testing the queue helped me understand practical usage in real-world systems like scheduling and task management.

Tack-05

Final Optimal Prompt:

write a python program for bubble sort and selection sort and maintain doctstring for every method

Code Screenshot :

```

1 from typing import List
2
3 def bubble_sort(arr: List[int]) -> None:
4     """Sort an array in-place using the bubble sort algorithm. Bubble sort repeatedly steps through the array, compares adjacent elements, and swaps them if they are in the wrong order. This process continues until no more swaps are needed, indicating the array is sorted.
5     Args:
6         arr: A list of integers to be sorted in ascending order (modified in-place).
7     Returns:
8         None. The array is sorted in-place.
9     Complexity:
10        Time: O(n^2) in all cases (best, average, worst).
11        Space: O(1) auxiliary space.
12    Example:
13        >>> data = [5, 2, 8, 1, 9]
14        >>> bubble_sort(data)
15        >>> data
16        [1, 2, 5, 8, 9]
17    """
18    n = len(arr)
19    for i in range(n):
20        # Flag to optimize: stop early if no swaps occur
21        swapped = False
22        for j in range(n - i - 1):
23            if arr[j] > arr[j + 1]:
24                arr[j], arr[j + 1] = arr[j + 1], arr[j]
25                swapped = True
26        if not swapped:
27            break
28
29 def selection_sort(arr: List[int]) -> None:
30     """Sort an array in-place using the selection sort algorithm. Selection sort divides the array into a sorted portion and an unsorted portion. It repeatedly finds the minimum element from the unsorted portion, places it at the end of the sorted portion, and expands the sorted region.
31     Args:
32         arr: A list of integers to be sorted in ascending order (modified in-place).
33     Returns:
34         None. The array is sorted in-place.
35     Complexity:
36        Time: O(n^2) in all cases (best, average, worst).
37        Space: O(1) auxiliary space.
38    Example:
39        >>> data = [5, 2, 8, 1, 9]
40        >>> selection_sort(data)
41        >>> data
42        [1, 2, 5, 8, 9]
43    """
44    n = len(arr)
45    for i in range(n):
46        # Find the index of the minimum element in the unsorted part
47        min_idx = i
48        for j in range(i + 1, n):
49            if arr[j] < arr[min_idx]:
50                min_idx = j
51        # Swap the minimum element with the current position
52        arr[i], arr[min_idx] = arr[min_idx], arr[i]
53
54 def _demo() -> None:
55     """Demonstrate and test the bubble_sort and selection_sort functions."""
56     test_cases = [
57         [5, 2, 8, 1, 9],
58         [3, 3, 3],
59         [1],
60         [100, 50, 25, 75, 10]
61     ]
62     for i, arr in enumerate(test_cases):
63         arr_bubble = arr.copy()
64         arr_selection = arr.copy()
65         bubble_sort(arr_bubble)
66         selection_sort(arr_selection)
67         print(f"Test {i + 1}:")
68         print(f"Original: {arr}")
69         print(f"Bubble: {arr_bubble}")
70         print(f"Selection: {arr_selection}")
71         print(f"Match: {arr_bubble == arr_selection}")
72         print()
73
74 if __name__ == "__main__":
75     _demo()

```

Output Screenshot:

```

• PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:: cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59265' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\sorts.py'
Test 1:
Original:    [5, 2, 8, 1, 9]
Bubble:     [1, 2, 5, 8, 9]
Selection:  [1, 2, 5, 8, 9]
Match: True

Test 2:
Original:    [3, 3, 3]
Bubble:     [3, 3, 3]
Selection:  [3, 3, 3]
Match: True

Test 3:
Original:    [1]
Bubble:     [1]
Selection:  [1]
Match: True

Test 4:
Original:    []
Bubble:     []
Selection:  []
Match: True

Test 5:
Original:    [100, 50, 25, 75, 10]
Bubble:     [10, 25, 50, 75, 100]
Selection:  [10, 25, 50, 75, 100]
Match: True

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

Executing both sorting algorithms helped me see how they differ step-by-step. Bubble Sort compares adjacent elements repeatedly, and I noticed how many swaps it performs even for

nearly-sorted lists. Selection Sort selected the smallest element in each pass and swapped it only once per iteration, making it seem more stable in terms of number of swaps. Through AI-generated comments and complexity docstrings, I understood why both algorithms are $O(n^2)$ but behave differently in execution. This comparison gave me clarity on where each algorithm is used and why they are not preferred for large data.
